

DS Lecture 4: Multicast

September 24th, 2025

Instructor: Elli Anastasiadi

`ellia@cs.aau.dk`

DEIS
Aalborg University
Denmark



AALBORG UNIVERSITY
DENMARK

I was gonna tell you guys a joke about UDP, but you might not get it.

- *The TCP Stack*



AALBORG UNIVERSITY
DENMARK



From last lectures

- ▶ We saw the issue with re-ordered messages over network, and how counting time in a distributed way is tricky.
- ▶ Lamport diagrams, [happens-before](#) .



What is it?

What is multicast?

A **multicast** is **one-to-many** communication between a single process and a **specific group** of processes such that **all members of the group receive the message**



What is it?

Examples

- ▶ Algorithms with failover/replication/redundancy
 - ▶ DNS
 - ▶ Databases
 - ▶ Banks!
- ▶ One-to-many
 - ▶ Streaming of TV/Radio
 - ▶ Industrial Systems
- ▶ Many-to-many
 - ▶ Skype, Teams, TikTok
 - ▶ MMOs

Agenda



Multicast problems and model

Implementing the ordered multicast

Totally Ordered Multicast

Logical time, causal broadcast
Vector Clocks

Disclaimer



- ▶ We assume closed and static groups, and
- ▶ Not discussing multiple groups.
- ▶ Not assuming hardware support.

Good news : All algorithms shown work both in sync and async networks

Requirements

Multicast



Assuming

- ▶ Reliable 1:1 communication (no hardware support).
- ▶ Sender might crash
- ▶ No order

Guarantees we want

- ▶ if a message is sent, it is delivered exactly once
- ▶ messages are eventually delivered to non-crashed (i.e. correct) processes
- ▶ fault tolerance (if one node fails, rest can carry on)

General broadcast structure



Instead of simple send and receive we get:

General broadcast structure

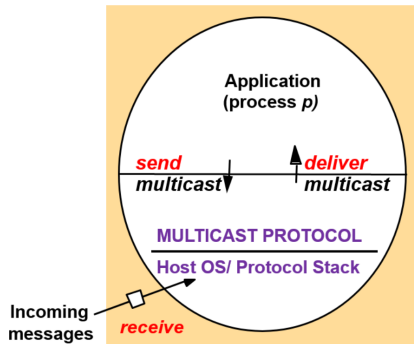
Instead of simple send and receive we get:

- ▶ Node 1 *broadcasts to*
broadcast algorithm
- ▶ broadcast algorithm
sends/receives from network
- ▶ network *sends/receives to*
broadcast algorithm
- ▶ broadcast algorithm *delivers to*
Node 2

General broadcast structure

Instead of simple send and receive we get:

- ▶ Node 1 *broadcasts to*
broadcast algorithm
- ▶ broadcast algorithm
sends/receives from network
- ▶ network *sends/receives to*
broadcast algorithm
- ▶ broadcast algorithm *delivers to*
Node 2



Conection to lecture 2



Conection to lecture 2



(Just so we are conenected to the literature terms)

- ▶ We have [best-effort](#) instead of fair-loss.
- ▶ We have [reliable](#) instead of perfect.
- ▶ We will be asuming async or partialy async network timing model (no bound on latency).

Problems

IP Multicast

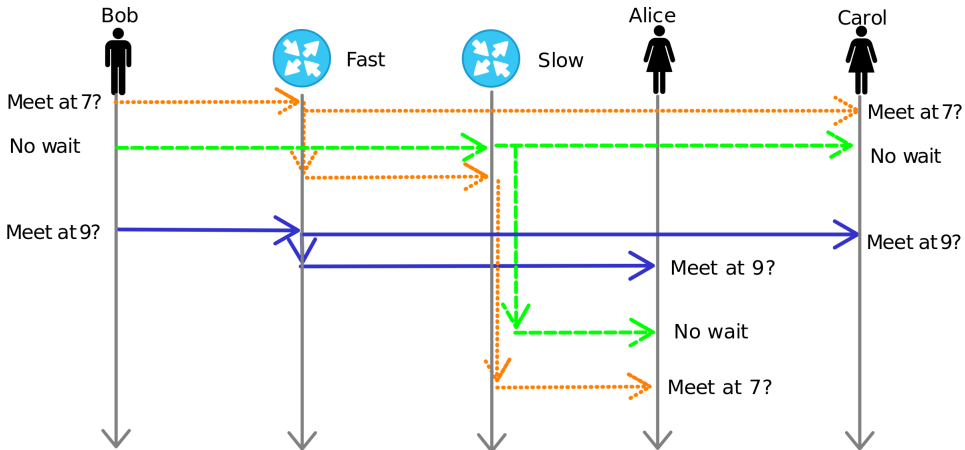


UDP has no guarantees

- ▶ No re-transmission
 - ▶ no reception guaranteed
 - ▶ one attempt only
- ▶ No ordering
 - ▶ Messages are delivered in arbitrary order

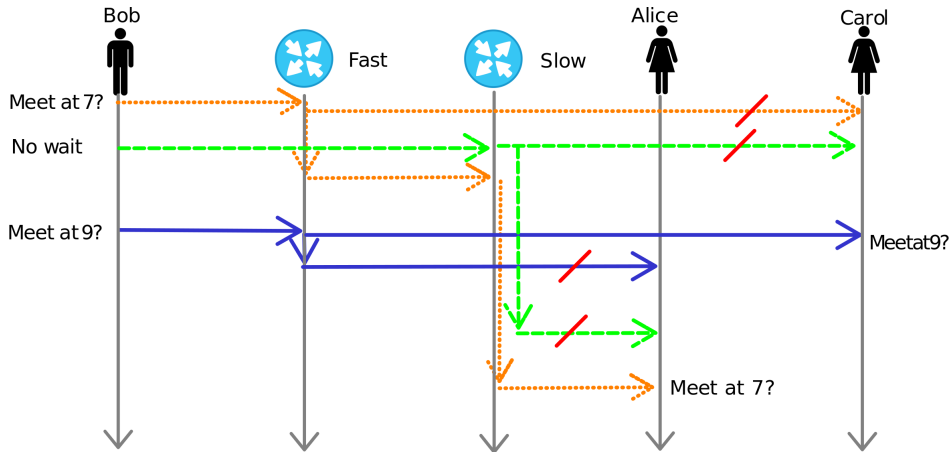
Problems

IP Multicast



Problems

IP Multicast



Implementing reliable broadcast algorithms



Implementing reliable broadcast algorithms



Not all broadcast algorithms are the same.

Implementing reliable broadcast algorithms

Not all broadcast algorithms are the same.

- **FIFO broadcast:** If m_1 and m_2 are broadcast by the same node, and $\text{broadcast}(m_1) \implies \text{broadcast}(m_2)$ then m_1 must be delivered before m_2 .

Implementing reliable broadcast algorithms

Not all broadcast algorithms are the same.

- ▶ **FIFO broadcast:** If m_1 and m_2 are broadcast by the same node, and $\text{broadcast}(m_1) \implies \text{broadcast}(m_2)$ then m_1 must be delivered before m_2 .
- ▶ **Causal broadcast:** If $\text{broadcast}(m_1) \implies \text{broadcast}(m_2)$ then m_1 must be delivered before m_2 .

Implementing reliable broadcast algorithms

Not all broadcast algorithms are the same.

- ▶ **FIFO broadcast:** If $m1$ and $m2$ are broadcast by the same node, and $broadcast(m1) \implies broadcast(m2)$ then $m1$ must be delivered before $m2$.
- ▶ **Causal broadcast:** If $broadcast(m1) \implies broadcast(m2)$ then $m1$ must be delivered before $m2$.
- ▶ **Total order broadcast:** If $m1$ is delivered before $m2$ on one node, then $m1$ must be delivered before $m2$ on all nodes.

Implementing reliable broadcast algorithms

Not all broadcast algorithms are the same.

- ▶ **FIFO broadcast:** If m_1 and m_2 are broadcast by the same node, and $\text{broadcast}(m_1) \implies \text{broadcast}(m_2)$ then m_1 must be delivered before m_2 .
- ▶ **Causal broadcast:** If $\text{broadcast}(m_1) \implies \text{broadcast}(m_2)$ then m_1 must be delivered before m_2 .
- ▶ **Total order broadcast:** If m_1 is delivered before m_2 on one node, then m_1 must be delivered before m_2 on all nodes.
- ▶ **FIFO-total order broadcast:** Combination of FIFO broadcast and total order broadcast

Ordered Multicast

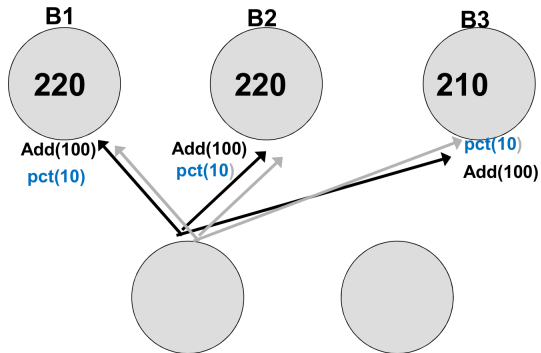


Figure: Unordered Multicast

Ordered Multicast

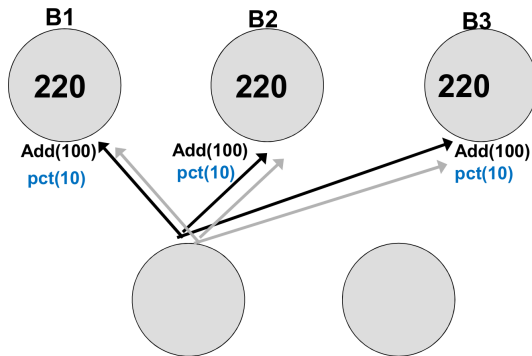


Figure: FIFO Multicast

Ordered Multicast

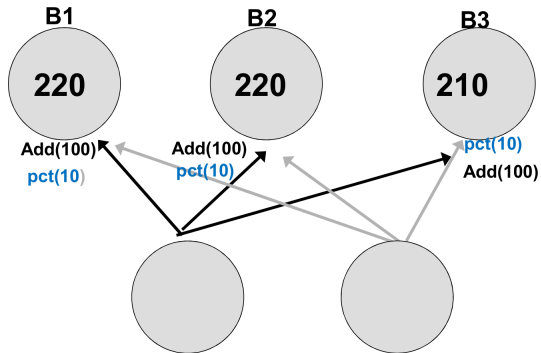


Figure: FIFO Multicast - Fails too

Ordered Multicast

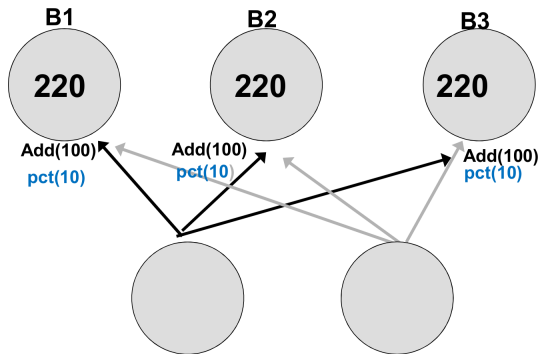


Figure: Total Order - good enough?

Ordered Multicast

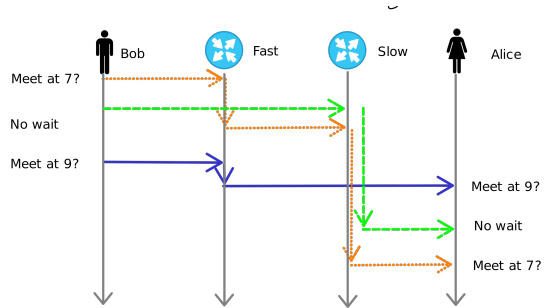
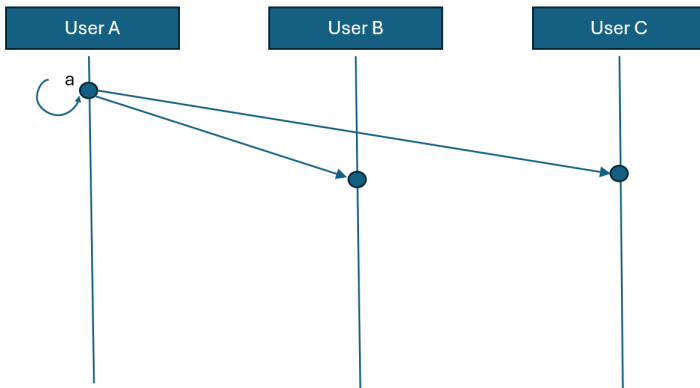
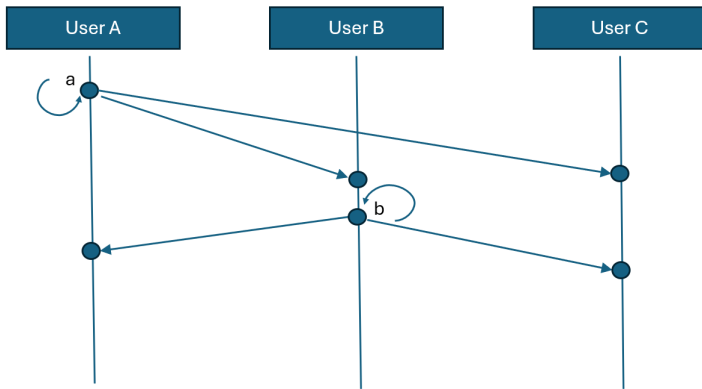


Figure: Alice gets the messages in a total order!

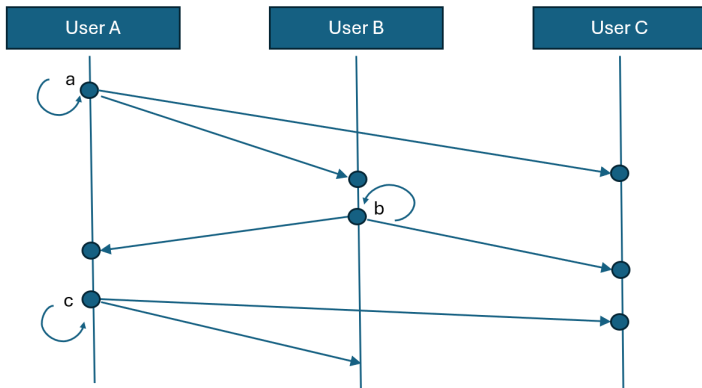
Example on FIFO broadcast



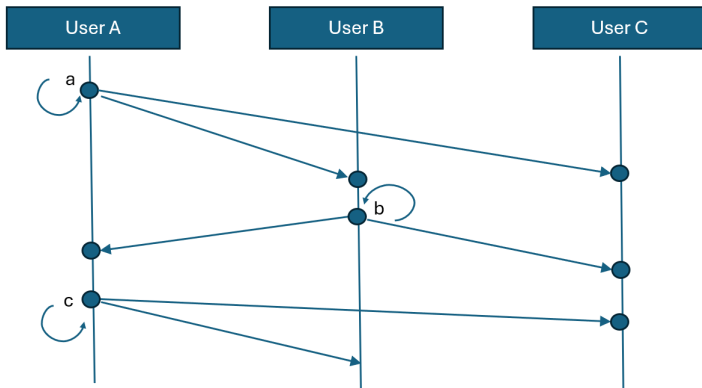
Example on FIFO broadcast



Example on FIFO broadcast

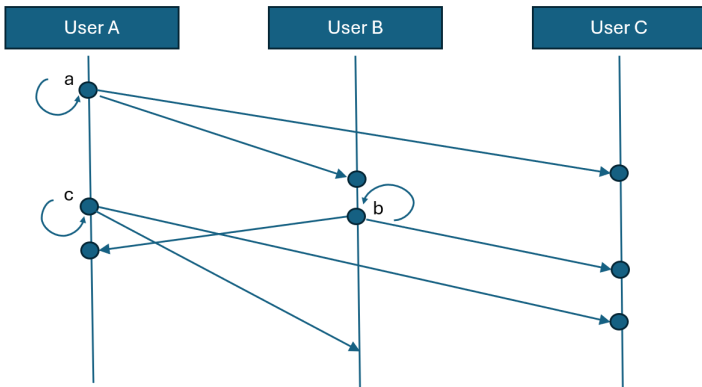


Example on FIFO broadcast

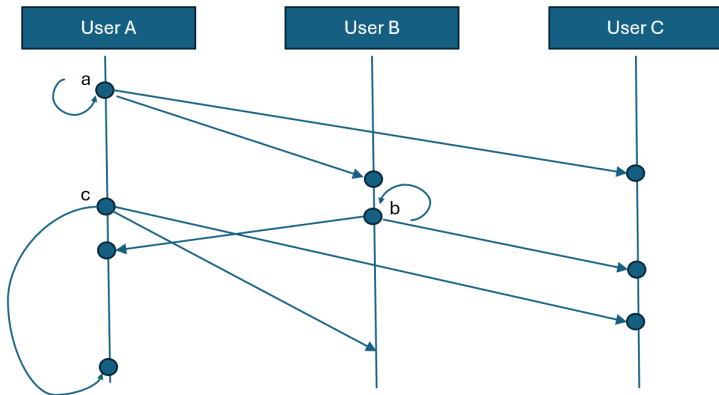


The messages from User A are always delivered in the order they were sent.

Causal broadcast



Total Order Broadcast



Hierarchy



Overall:

Hierarchy



Overall:

- ▶ **Best-effort** broadcast is the bare minimum, we enhance it with retransmission to get:

Hierarchy



Overall:

- ▶ **Best-effort** broadcast is the bare minimum, we enhance it with retransmission to get:
- ▶ **Reliable** broadcast which makes not guarantees on the order. Then, we have

Hierarchy



Overall:

- ▶ **Best-effort** broadcast is the bare minimum, we enhance it with retransmission to get:
- ▶ **Reliable** broadcast which makes not guarantees on the order. Then, we have
- ▶ **FIFO** broadcast which does not re-order same origin messages, then

Hierarchy

Overall:

- ▶ **Best-effort** broadcast is the bare minimum, we enhance it with retransmission to get:
- ▶ **Reliable** broadcast which makes not guarantees on the order. Then, we have
- ▶ **FIFO** broadcast which does not re-order same origin messages, then
- ▶ **Causal** broadcast which does not re-order happens-before related messages, then

Hierarchy

Overall:

- ▶ **Best-effort** broadcast is the bare minimum, we enhance it with retransmission to get:
- ▶ **Reliable** broadcast which makes not guarantees on the order. Then, we have
 - ▶ **FIFO** broadcast which does not re-order same origin messages, then
 - ▶ **Causal** broadcast which does not re-order happens-before related messages, then
 - ▶ **Total order** broadcast which delivers to all processes in the same order.

Hierarchy

Overall:

- ▶ **Best-effort** broadcast is the bare minimum, we enhance it with retransmission to get:
- ▶ **Reliable** broadcast which makes not guarantees on the order. Then, we have
 - ▶ **FIFO** broadcast which does not re-order same origin messages, then
 - ▶ **Causal** broadcast which does not re-order happens-before related messages, then
 - ▶ **Total order** broadcast which delivers to all processes in the same order.
- ▶ **FIFO-Total order** broadcast which combines all.

Implementations



We will have two layers.

Implementations



We will have two layers.
Step 1: Make reliable

Implementations



We will have two layers.

Step 1: Make reliable

Step 2: Enforcing delivery orders.

Implementations



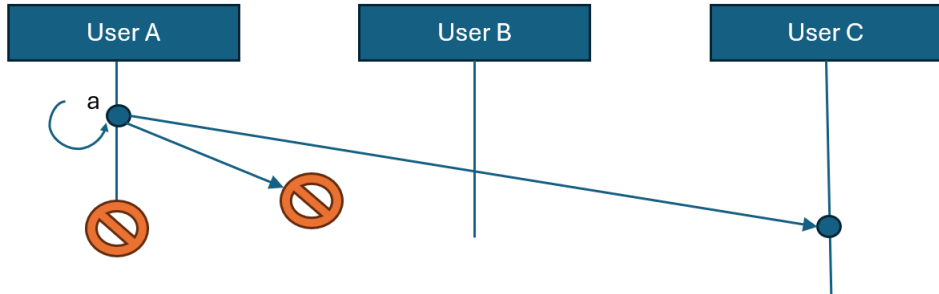
We will have two layers.

Step 1: Make reliable

Step 2: Enforcing delivery orders.

Brainstorm session: Use point to point links and describe protocol.

Problem 1



Reliable Multicast

Properties



- ▶ Integrity
 - ▶ Messages are delivered at most once.
- ▶ Validity
 - ▶ A process delivers to itself (or crashes).
- ▶ Agreement
 - ▶ All correct processes deliver the same messages.

Reliable Multicast



- ▶ Integrity
- ▶ Validity
- ▶ Agreement

Reliable Multicast



- ▶ Integrity
 - ▶ Yes
- ▶ Validity
- ▶ Agreement

Reliable Multicast



- ▶ Integrity
 - ▶ Yes
- ▶ Validity
 - ▶ Yes
- ▶ Agreement

Reliable Multicast



- ▶ Integrity
 - ▶ Yes
- ▶ Validity
 - ▶ Yes
- ▶ Agreement
 - ▶ Yes

Reliable Multicast



- ▶ Integrity
 - ▶ Yes
- ▶ Validity
 - ▶ Yes
- ▶ Agreement
 - ▶ Yes
- ▶ 1 multicast = $O(N^2)$ messages in the network

Reliable Multicast

- ▶ Integrity
 - ▶ Yes
- ▶ Validity
 - ▶ Yes
- ▶ Agreement
 - ▶ Yes
- ▶ 1 multicast = $O(N^2)$ messages in the network

Decrease complexity via: Gossip (for example). Each deliver triggers three random sends. ($O(n)$ messages, works with high probability).

Ordered Multicast



- ▶ FIFO Ordered
 - ▶ Messages from p_n are received at p_k in order sent by p_n
- ▶ Total Ordered
 - ▶ All messages are received in same order at p_n and p_k
- ▶ Causally Ordered
 - ▶ If m_1 happened-before m_2 , then p_n receives m_1 before m_2



Details and implications

1. FIFO ordering and causal ordering are only partial orderings:
 - ▶ not all messages are sent by the same process, in general;
 - ▶ some multicasts are concurrent (not ordered by happened-before)
2. Reliable totally ordered multicast is also referred to in the literature as an atomic multicast.
3. In principle, ordered multicast do not imply reliability:
 - ▶ under total ordering, if correct process p delivers message m and then delivers m' , then a correct process q can deliver m without also delivering m' or any other message ordered after m

FIFO broadcast



FIFO broadcast



Reliable Multicast over IP is FIFO
We respect sequence-numbers of sender!

FIFO broadcast

Reliable Multicast over IP is FIFO

We respect sequence-numbers of sender!

on initialisation **do**

$sendSeq := 0$; $delivered := \langle 0, 0, \dots, 0 \rangle$; $buffer := \{\}$

end on

on request to broadcast m at node N_i **do**

send $(i, sendSeq, m)$ via reliable broadcast

$sendSeq := sendSeq + 1$

end on

on receiving msg from reliable broadcast at node N_i **do**

$buffer := buffer \cup \{msg\}$

while $\exists sender, m. (sender, delivered[sender], m) \in buffer$ **do**

deliver m to the application

$delivered[sender] := delivered[sender] + 1$

end while

end on

Break

Totally Ordered Multicast



Totally Ordered Multicast



Things are a bit more tricky: somehow we must "pick" the order.



Totally Ordered Multicast

Things are a bit more tricky: somehow we must "pick" the order.

Ideas:

- ▶ do as FIFO, but only one sequence-number,
- ▶ each message has a unique id/hash,
- ▶ agree globally on "next"-message
 1. Use global sequencer (leader),

Totally Ordered Multicast

Things are a bit more tricky: somehow we must "pick" the order.

Ideas:

- ▶ do as FIFO, but only one sequence-number,
- ▶ each message has a unique id/hash,
- ▶ agree globally on "next"-message
 1. Use global sequencer (leader),
 2. Problem: leader crashes = same issue as reliable broadcast.
 3. Changing leaders nicely is hard, or

Totally Ordered Multicast

Things are a bit more tricky: somehow we must "pick" the order.

Ideas:

- ▶ do as FIFO, but only one sequence-number,
- ▶ each message has a unique id/hash,
- ▶ agree globally on "next"-message
 1. Use global sequencer (leader),
 2. Problem: leader crashes = same issue as reliable broadcast.
 3. Changing leaders nicely is hard, or
 4. happens-before approach (see later for algorithm).
 5. Does not work if some process crashes, or

Totally Ordered Multicast

Things are a bit more tricky: somehow we must "pick" the order.

Ideas:

- ▶ do as FIFO, but only one sequence-number,
- ▶ each message has a unique id/hash,
- ▶ agree globally on "next"-message
 1. Use global sequencer (leader),
 2. Problem: leader crashes = same issue as reliable broadcast.
 3. Changing leaders nicely is hard, or
 4. happens-before approach (see later for algorithm).
 5. Does not work if some process crashes, or
 6. use negotiation (ISIS) - Takes more rounds.

Totally Ordered Multicast (Sequencer)



We need: Leader election + reliable links.



Totally Ordered Multicast (Sequencer)

We need: Leader election + reliable links.
Leader broadcasts in FIFO.

Problems

- ▶ Sequencer is bottle-neck
- ▶ Single point of failure

Totally Ordered Multicast (ISIS)

Idea: Negotiate next ID

1. Process p broadcasts message m
2. Every other process q responds to p with a proposal
3. p picks largest proposed value, broadcasts

The Trick

Track “largest proposed value” and “largest agreed value” at each process

Is it totally ordered?

- ▶ A node receives a message to vote on, it sends out a proposal for it, and it inserts the message into the holdback queue with the proposal value and the status as *non-agreed*
- ▶ The node receives the agreed-on value for the message, and it reinserts the message with the agreed-on value and the status as *agreed*
- ▶ The node orders the messages in the queue using their values, and it delivers them in order, but only as long as they are *agreed*
 - ▶ In other words, a message with a proposal lower than agreed messages, will block following agreed messages from being delivered

Is it totally ordered?

- ▶ Let us consider processes A and B, and messages m and n
- ▶ By executing the protocol, A assigns agreed-upon timestamp 1 to m and 2 to n, and it delivers m before n
- ▶ Let us consider that B delivers n before m?
 - ▶ Let us imagine that, based on received Order messages, B thinks m and n timestamps are different from 1 and 2 respectively. Not possible, since the agreed-on timestamps were sent in multicast to everybody, and they are 1 and 2 respectively.
 - ▶ Let us imagine that B has given a proposed timestamp of 3 to m, to deliver n that has agreed-upon timestamp 2. Not possible, the agreed-upon timestamp of m would end up being ≥ 3 , and it must be 1.
 - ▶ Let us imagine that B knows nothing about m, and n has already its agreed-upon timestamp of 2 thus B wants to deliver it. Not possible since B would later vote for a timestamp larger than 2 for m, leading to a agreed-upon timestamp > 2 , but it must be 1.

Totally Ordered Multicast (ISIS)

- ▶ Good:
 - ▶ Sequence numbers are monotonically increasing
 - ▶ Nobody will deliver “early”
- ▶ Bad: every broadcast requires negotiation (3 rounds)
 - ▶ Sequencer has 2 rounds

Repetition (fast)



Repetition (fast)



- From last lecture: physical clock timestamps might not respect causality.

Repetition (fast)

- From last lecture: physical clock timestamps might not respect causality.

Solution: Logical clocks!

Repetition (fast)

- From last lecture: physical clock timestamps might not respect causality.

Solution: Logical clocks!

- designed to capture causal dependencies (happens-before relation):

$$(e_1 \implies e_2) \text{ then } (T(e_1) < T(e_2))$$

Repetition (fast)

- From last lecture: physical clock timestamps might not respect causality.

Solution: Logical clocks!

- designed to capture causal dependencies (happens-before relation):

$$(e_1 \implies e_2) \text{ then } (T(e_1) < T(e_2))$$

We will look at two types of logical clocks

1. Lamport Clocks, and
2. Vector clocks

Lamport clocks Algorithm



Lamport clocks Algorithm

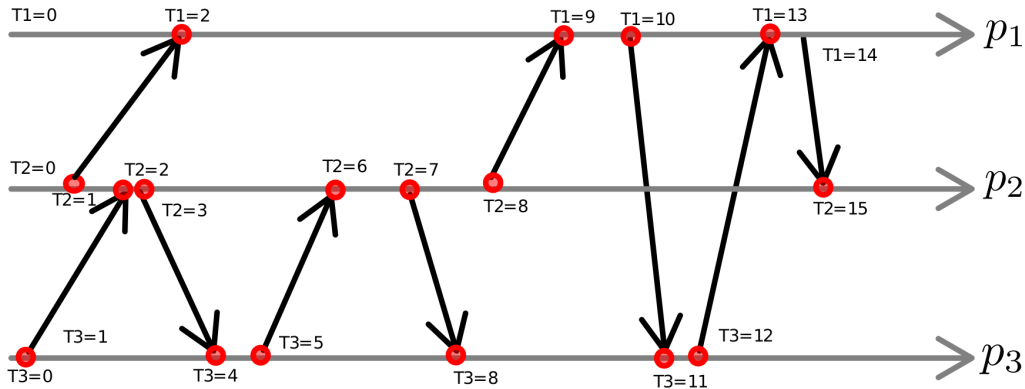


- ▶ Each node/process gets its own local clock variable t , and initializes at 0.
- ▶ Every time a node gets an event they increment t .
- ▶ If the event is a send of message m , the node sends (t, m) (after incrementing t).
- ▶ If the node is receiving a message then the node updates their clock to the maximum of their local clock and the received value, and then increments.

Lamport clocks



Lamport clocks



Totally order broadcast via Lamport clocks





Totally order broadcast via Lamport clocks

- ▶ Attach some kind of logical timestamp to all messages
- ▶ Deliver messages in the order of the timestamps



Totally order broadcast via Lamport clocks

- ▶ Attach some kind of logical timestamp to all messages
- ▶ Deliver messages in the order of the timestamps
- ▶ Problem: I have to deliver message of timestamp 5 how can I know in the future I will not receive message with timestamp 4? Brainstorm here.

Totally order broadcast via Lamport clocks

- ▶ Attach some kind of logical timestamp to all messages
- ▶ Deliver messages in the order of the timestamps
- ▶ Problem: I have to deliver message of timestamp 5 how can I know in the future I will not receive message with timestamp 4? Brainstorm here.
- ▶ Use FIFO links and wait for timestamp more than 5 from every node

Causal broadcast via logical clocks





Causally Ordered Multicast

Idea

- ▶ Order events by [happened-before](#) relationship,
- ▶ Use “vectored” not-quite-Lamport clocks (Vector Clocks), and
- ▶ Track only “send” as an event.



Vector Clocks

Not-quite-Lamport clocks, and they are vectors

- ▶ keep track of “last known time” of other processes
- ▶ “gossip” about “last known time” during communication

Vector Clocks

Let V_i be the vector of process $p_i \in \{p_0, \dots, p_n\}$, then

- ▶ initially $V_i[j] = 0$ for all $j \in 0 \dots n$,
- ▶ before event $V'_i[i] = V_i[i] + 1$,
- ▶ attach V to any message sent,
- ▶ on receive of V' we let $V_i[j] = \max(V_i[j], V'[j])$ for $j \in 0 \dots n$.

Vector Clocks

Given two vectors V and W ,

- ▶ $V = W$ if all values match,
 - ▶ for all $j \in 0 \dots n$, $V[j] = W[j]$
- ▶ $V \leq W$ if all values in V are less than or equal those in W ,
 - ▶ for all $j \in 0 \dots n$, $V[j] \leq W[j]$
- ▶ $V < W$ if all values in V are less than or equal to W and $W \neq V$
 - ▶ $V \leq W$ and $V \neq W$

Vector Clocks

Real vector clocks for CO Multicast update only on send operations!

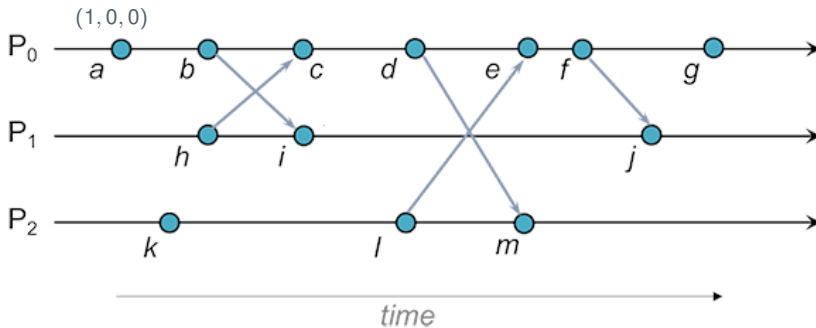


Figure: Does *l* happen between *h* and *i*?

Vector Clocks

Real vector clocks for CO Multicast update only on send operations!

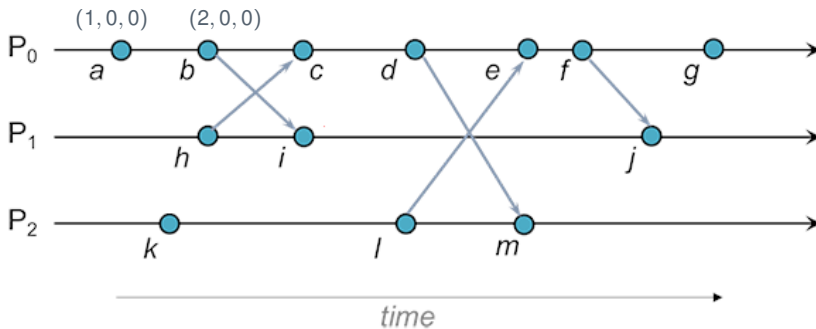


Figure: Does l happen between h and i ?

Vector Clocks

Real vector clocks for CO Multicast update only on send operations!

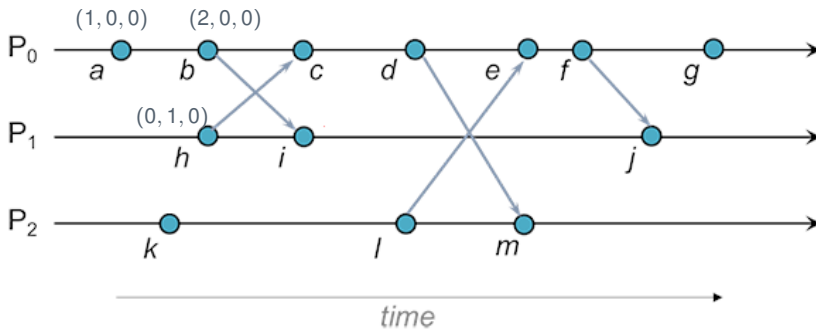


Figure: Does l happen between h and i ?

Vector Clocks

Real vector clocks for CO Multicast update only on send operations!

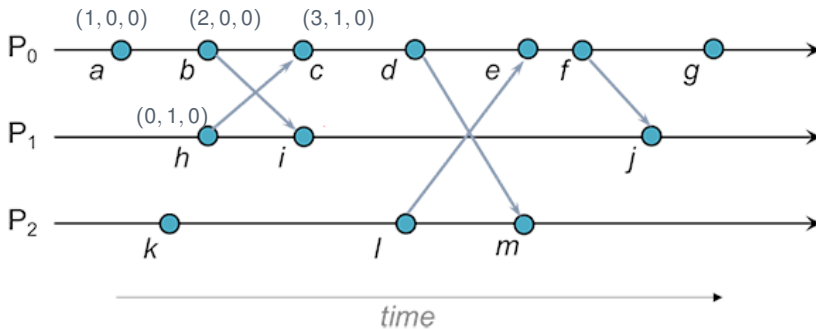


Figure: Does l happen between h and i ?

Vector Clocks

Real vector clocks for CO Multicast update only on send operations!

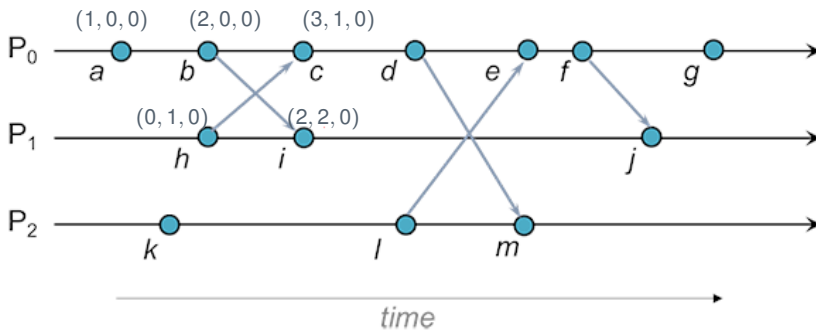


Figure: Does l happen between h and i ?

Vector Clocks

Real vector clocks for CO Multicast update only on send operations!

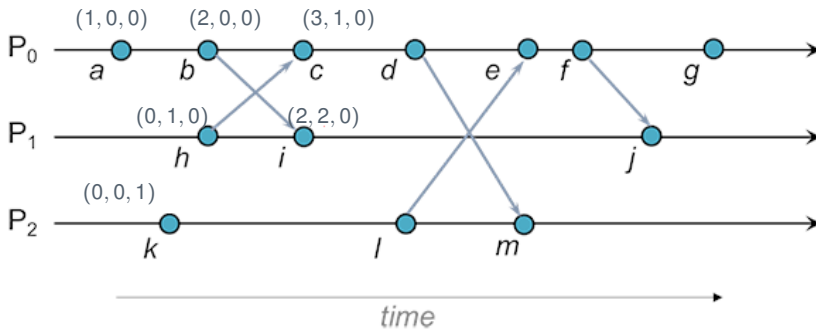


Figure: Does l happen between h and i ?

Vector Clocks

Real vector clocks for CO Multicast update only on send operations!

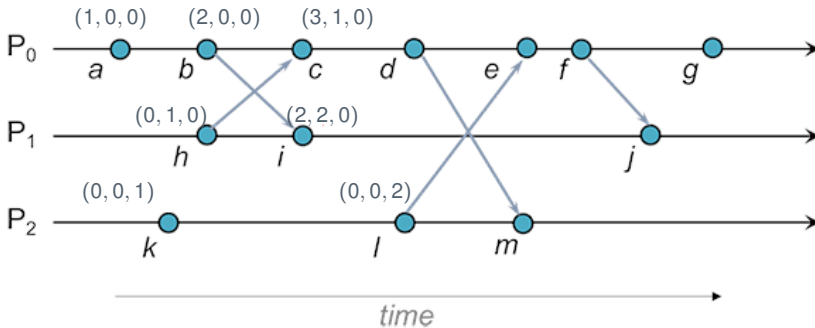


Figure: Does l happen between h and i ?

Vector Clocks

Real vector clocks for CO Multicast update only on send operations!

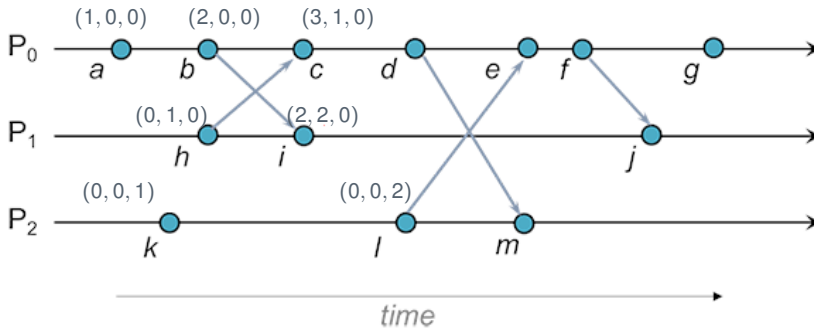


Figure: l happens concurrently with h and i

Vector Clocks, as used for CO Multicast

Delivery of a message m sent from P_i to P_j , and V_j update, are delayed until both:

1. $V_m[i] = V_j[i] + 1$ (V_m is the clock on the message)
2. $V_m[k] \leq V_j[k]$ for all $k \neq i$

P_0 • $V_0 = (0, 0, 0)$

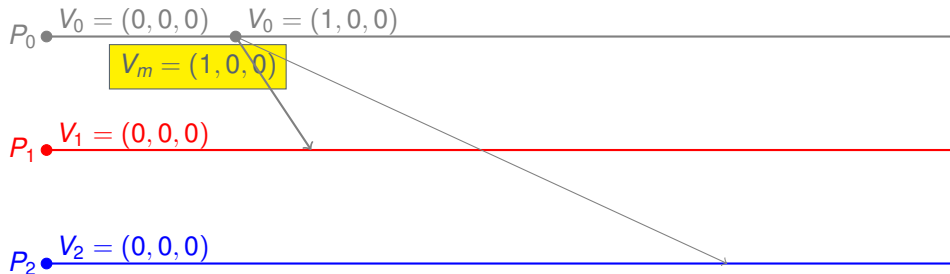
P_1 • $V_1 = (0, 0, 0)$

P_2 • $V_2 = (0, 0, 0)$

Vector Clocks, as used for CO Multicast

Delivery of a message m sent from P_i to P_j , and V_j update, are delayed until both:

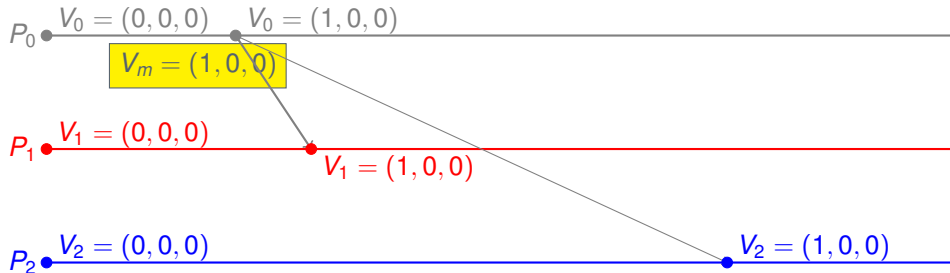
1. $V_m[i] = V_j[i] + 1$ (V_m is the clock on the message)
2. $V_m[k] \leq V_j[k]$ for all $k \neq i$



Vector Clocks, as used for CO Multicast

Delivery of a message m sent from P_i to P_j , and V_j update, are delayed until both:

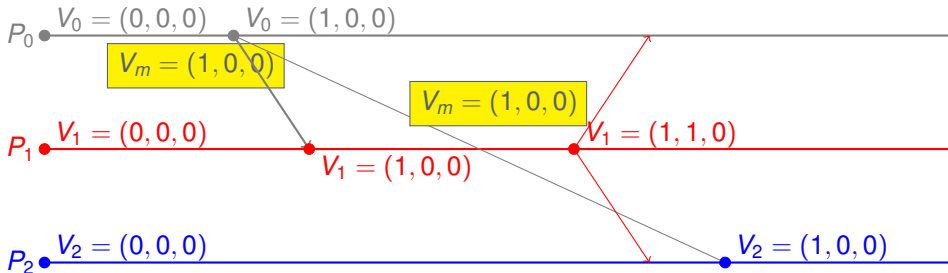
1. $V_m[i] = V_j[i] + 1$ (V_m is the clock on the message)
2. $V_m[k] \leq V_j[k]$ for all $k \neq i$



Vector Clocks, as used for CO Multicast

Delivery of a message m sent from P_i to P_j , and V_j update, are delayed until both:

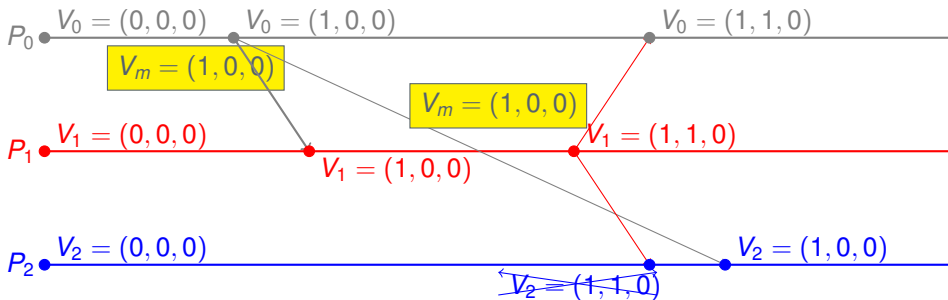
1. $V_m[i] = V_j[i] + 1$ (V_m is the clock on the message)
2. $V_m[k] \leq V_j[k]$ for all $k \neq i$



Vector Clocks, as used for CO Multicast

Delivery of a message m sent from P_i to P_j , and V_j update, are delayed until both:

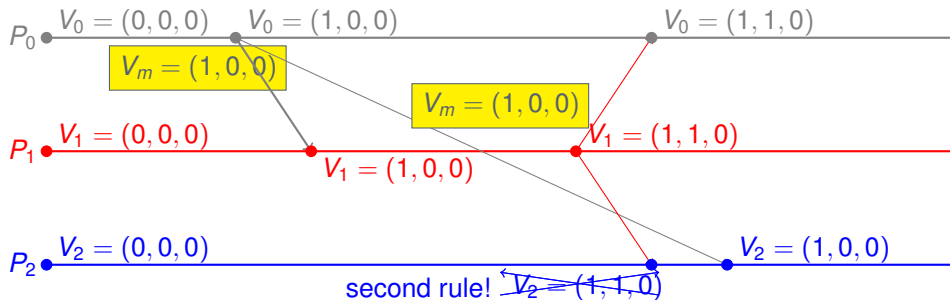
1. $V_m[i] = V_j[i] + 1$ (V_m is the clock on the message)
2. $V_m[k] \leq V_j[k]$ for all $k \neq i$



Vector Clocks, as used for CO Multicast

Delivery of a message m sent from P_i to P_j , and V_j update, are delayed until both:

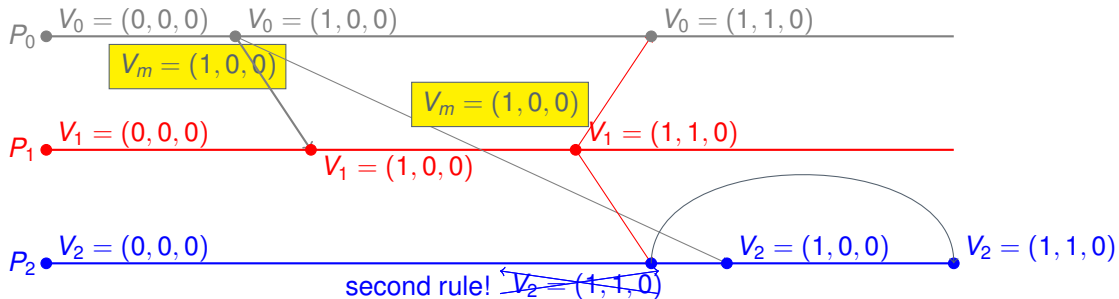
1. $V_m[i] = V_j[i] + 1$ (V_m is the clock on the message)
2. $V_m[k] \leq V_j[k]$ for all $k \neq i$



Vector Clocks, as used for CO Multicast

Delivery of a message m sent from P_i to P_j , and V_j update, are delayed until both:

1. $V_m[i] = V_j[i] + 1$ (V_m is the clock on the message)
2. $V_m[k] \leq V_j[k]$ for all $k \neq i$





Lesson of Today

- ▶ Multicast is notoriously difficult
 - ▶ be careful of homebrewed solutions
- ▶ All algorithms shown work in async setting

Not discussed

- ▶ Many can be combined/layered
 - ▶ reliable causally ordered multicast over IP
- ▶ Multiple multicast groups = extra complexity
- ▶ Totally Ordered Reliable mutlticast = impossible in async setting